L-QLSTM Fleet Emissions Forecaster

Based on: *Linear-Layer-Enhanced QLSTM* (Cao et al.)

Paper Relevance

Paper (Carbon Price)	Our Problem (Fleet Emissions)
EU ETS time-series	Fleet diesel/energy time-series
5 OHLCV features	20 features (4 streams x 5 machines)
1 output (price)	5 outputs (CO2e per machine)
QLSTM fails: 56% qubit waste	Same failure: 83% waste (20 feat, 4 qubits)
L-QLSTM fixes with shared Le	Same fix applied here
SCZ: all-to-all entanglement	Captures fuel/elec/heat correlations

Three Paper Innovations

1. **Shared Le before VQCs** (Eq.9) — compress to N_QUBITS, measure 100% of qubits
2. **Separate Li after each VQC** (Section 3.2) — gate-specific remapping
3. **SCZ variational layer** (Section 3.3, Fig.8) — CRZ all-to-all vs CNOT chain

Forward Pass (Equations 9-15)

```
v_t = [h_{t-1}, x_t]
z_t = Le(v_t)                          # Eq.9  shared linear embedding
f_t = sigmoid( L1( VQC1(z_t) ) )       # Eq.10 forget gate
i_t = sigmoid( L2( VQC2(z_t) ) )       # Eq.11 input gate
g_t = tanh( L3( VQC3(z_t) ) )          # Eq.12 cell gate
c_t = f_t * c_{t-1} + i_t * g_t       # Eq.13 cell state
o_t = sigmoid( L4( VQC4(z_t) ) )       # Eq.14 output gate
h_t = o_t * tanh(c_t)                  # Eq.15 hidden state
y_t = Lo(h_t)                           # Eq.16 output
```

Section 1 — Install and Import

In [1]:

```
import autoray
import pennylane
print(f"Autoray Location: {autoray.__file__}")
print(f"Autoray Version: {autoray.__version__}")
```

```
Autoray Location: c:\Users\RikteemBhowmick\Projects 2025\L-QSTM\qlstmvenv\Lib\site-packa
ges\autoray\__init__.py
Autoray Version: 0.6.12
```

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

In [2]:

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import matplotlib.dates as mdates
from datetime import timedelta
import warnings; warnings.filterwarnings('ignore')
import torch
import torch.nn as nn
from torch.utils.data import DataLoader, TensorDataset
import pennylane as qml
from sklearn.preprocessing import MinMaxScaler
from sklearn.metrics import mean_absolute_error, mean_squared_error
np.random.seed(42); torch.manual_seed(42)
print(f"PennyLane: {qml.__version__}  PyTorch: {torch.__version__}")
```

```
PennyLane: 0.38.0  PyTorch: 2.3.0+cpu
```

Section 2 — Synthetic Fleet Data

5 machines, 365 days. Patterns: weekday/weekend cycles, seasonal drift, maintenance every 45 days.

In [3]:

```
MACHINES = {
    'Excavator_A'  : dict(fuel=12.0, elec=3.5,  heat=18.0, rp=0.002, ra=0.05),
    'Generator_B'  : dict(fuel=8.0,  elec=0.5,  heat=10.0, rp=0.001, ra=0.03),
    'Crane_C'      : dict(fuel=9.5,  elec=5.0,  heat=8.0,  rp=0.003, ra=0.04),
    'Compressor_D' : dict(fuel=5.5,  elec=8.0,  heat=15.0, rp=0.002, ra=0.02),
    'Truck_Fleet'  : dict(fuel=15.0, elec=2.0,  heat=5.0,  rp=0.004, ra=0.06),
}
EF = {'fuel': 2.68, 'refrigerant': 2088.0, 'electricity': 0.82, 'heat': 0.27}
MACHINE_LIST = list(MACHINES.keys())

def generate_data(machines, n_days=365, start='2024-01-01'):
    rng = np.random.default_rng(42)
    dates = pd.date_range(start, periods=n_days, freq='D')
    records = []
    for mname, p in machines.items():
        maint = set(range(0, n_days, 45))
        for i, ts in enumerate(dates):
            wkend  = 0.35 if ts.dayofweek >= 5 else 1.0
            season = 1.0 + 0.12 * np.sin(2 * np.pi * ts.dayofyear / 365)
            mf     = 0.15 if i in maint else 1.0
            load   = rng.uniform(0.65, 1.0) * wkend * season * mf
            fuel   = max(0, p['fuel']*24*load + rng.normal(0, p['fuel']*2))
            elec   = max(0, p['elec']*24*load + rng.normal(0, p['elec']*1.2))
            heat   = max(0, p['heat']*24*load + rng.normal(0, p['heat']*1.0))
            refrig = p['ra'] if rng.random() < p['rp']*24 else 0.0
            co2e   = fuel*EF['fuel'] + refrig*EF['refrigerant'] + elec*EF['electricity']
            records.append(dict(date=ts, machine=mname,
                fuel_L=round(fuel,2), refrigerant_kg=round(refrig,4),
                electricity_kWh=round(elec,2), heat_MJ=round(heat,2), co2e_kg=round(co2e
    return pd.DataFrame(records)

df = generate_data(MACHINES)
print(f"Shape: {df.shape}")
df.head(3)
```

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

```
Shape: (1825, 7)
```

Out[3]:

	date	machine	fuel_L	refrigerant_kg	electricity_kWh	heat_MJ	co2e_kg
0	2024-01-01	Excavator_A	14.90	0.0	14.78	76.73	72.78
1	2024-01-02	Excavator_A	289.79	0.0	82.30	429.78	960.16
2	2024-01-03	Excavator_A	244.63	0.0	66.18	359.24	806.89

Section 3 — Preprocessing

Paper Section 4.1.1: min-max normalize [0,1], split 64/16/20%.

In [4]:

```
FEATURE_COLS = ['fuel_L','refrigerant_kg','electricity_kWh','heat_MJ']
LOOKBACK = 10

pX = df.pivot(index='date', columns='machine', values=FEATURE_COLS)
pX.columns = [f"{f}__{m}" for f,m in pX.columns]
py = df.pivot(index='date', columns='machine', values='co2e_kg')
py.columns = [f"co2e__{m}" for m in py.columns]
full = pd.concat([pX,py],axis=1).dropna()

fcols = [c for c in full.columns if not c.startswith('co2e')]
tcols = [c for c in full.columns if c.startswith('co2e')]

scaler_X = MinMaxScaler(); scaler_y = MinMaxScaler()
X_sc = scaler_X.fit_transform(full[fcols].values).astype(np.float32)
y_sc = scaler_y.fit_transform(full[tcols].values).astype(np.float32)

def make_seq(X, y, lb):
    Xs, ys = [], []
    for i in range(lb, len(X)):
        Xs.append(X[i-lb:i]); ys.append(y[i])
    return np.array(Xs, dtype=np.float32), np.array(ys, dtype=np.float32)

Xs, ys = make_seq(X_sc, y_sc, LOOKBACK)
s1=int(len(Xs)*0.64); s2=int(len(Xs)*0.80)
X_tr,y_tr = Xs[:s1],ys[:s1]
X_va,y_va = Xs[s1:s2],ys[s1:s2]
X_te,y_te = Xs[s2:],ys[s2:]
print(f"Train:{X_tr.shape}  Val:{X_va.shape}  Test:{X_te.shape}")
print(f"Features: {X_tr.shape[2]}  |  Targets: {y_tr.shape[1]}")
```

```
Train:(227, 10, 20)  Val:(57, 10, 20)  Test:(71, 10, 20)
Features: 20  |  Targets: 5
```

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

Section 4 — Quantum Device and SCZ Circuit

SCZ (Fig. 8 of paper) vs CNOT (Fig. 7):

- CNOT: chain connectivity, rotation AFTER entanglement
- CRZ all-to-all: every pair connected, rotation DURING entanglement
- Paper result: **34.83% RMSE reduction** with SCZ over CNOT

In [5]:

```
N_QUBITS = 4   # = hidden_size: ALL qubits measured, 0% waste
N_LAYERS = 2   # SCZ variational depth

# lightning.qubit: PennyLane's optimised C++ simulator — ~10x faster than
# qiskit.aer for small circuits.  Already installed via pennylane-lightning.
# diff_method='adjoint': computes gradients in a single forward+backward pass
# (O(params) cost) instead of parameter-shift's 2×params circuit evaluations.
# To switch to real IBM hardware later, swap only this device line — the rest
# of the model is unchanged.
dev = qml.device('lightning.qubit', wires=N_QUBITS)
print(f"Device: {dev.name}")

def scz_layer(weights, n_qubits):
    """
    SCZ layer (Fig. 8 of paper):
    1. RZ on every qubit  (pre-entanglement rotation)
    2. CRZ between ALL pairs  (all-to-all entanglement with rotation)
    3. RZ on every qubit  (post-entanglement rotation)
    """
    for i in range(n_qubits):
        qml.RZ(weights[i], wires=i)
    for i in range(n_qubits):
        for j in range(i+1, n_qubits):
            qml.CRZ(weights[i], wires=[i,j])
    for i in range(n_qubits):
        qml.RZ(weights[n_qubits+i], wires=i)

@qml.qnode(dev, interface='torch', diff_method='adjoint')
def vqc_scz(inputs, weights):
    """
    VQC with SCZ — replaces one LSTM gate.
    inputs : (N_QUBITS,) from Le embedding
    weights: (N_LAYERS, N_QUBITS*2)
    returns: N_QUBITS Pauli-Z values in [-1,1]
    ALL qubits measured (Le ensures all carry information)
    """
    qml.AngleEmbedding(inputs, wires=range(N_QUBITS), rotation='Y')
    for l in range(N_LAYERS):
        scz_layer(weights[l], N_QUBITS)
    return [qml.expval(qml.PauliZ(w)) for w in range(N_QUBITS)]

print("\nSCZ-VQC circuit:")
print(qml.draw(vqc_scz)(np.zeros(N_QUBITS), np.zeros((N_LAYERS, N_QUBITS*2))))
print(f"\nMeasured: {N_QUBITS}/{N_QUBITS} = 100%  (vanilla QLSTM would measure {N_QUBITS
```

```
Device: lightning.qubit
```

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

```
SCZ-VQC circuit:
0: ─╭AngleEmbedding(M0)──RZ(0.00)─╭●────────╭●────────╭●────────RZ(0.00)──RZ(0.00)──────
──────
1: ─├AngleEmbedding(M0)──RZ(0.00)─╰RZ(0.00)─│─────────│─────────╭●────────╭●────────RZ
(0.00)
2: ─├AngleEmbedding(M0)──RZ(0.00)──────────╰RZ(0.00)─│─────────╰RZ(0.00)─│─────────╭●──
──────
3: ─╰AngleEmbedding(M0)──RZ(0.00)──────────────────╰RZ(0.00)──────────╰RZ(0.00)─╰RZ
(0.00)

────────────╭●────────╭●────────╭●────────RZ(0.00)──────────────────────────────┤  <Z>
──RZ(0.00)─╰RZ(0.00)─│─────────│─────────╭●────────╭●────────RZ(0.00)──────────┤  <Z>
──RZ(0.00)──RZ(0.00)─╰RZ(0.00)─│─────────╰RZ(0.00)─│─────────╭●────────RZ(0.00)─┤  <Z>
──RZ(0.00)──RZ(0.00)──────────╰RZ(0.00)──────────╰RZ(0.00)─╰RZ(0.00)──RZ(0.00)─┤  <Z>

M0 =
[0. 0. 0. 0.]

Measured: 4/4 = 100%  (vanilla QLSTM would measure 4/24 = 16%)
```

Section 5 — L-QLSTM Cell (Equations 9-15)

In [6]:

```python
class LQLSTMCell(nn.Module):
    """
    L-QLSTM Cell from Cao et al.

    Key design (Section 3.1 & 3.2):
    - Shared Le: ONE linear layer for all 4 VQCs (parameter sharing, cross-gate dependen
    - Separate Li: FOUR linear layers after VQCs (each gate has different function)
    - SCZ VQC: all-to-all entanglement (Section 3.3)
    """
    def __init__(self, input_size, hidden_size):
        super().__init__()
        self.hidden_size = hidden_size
        combined = input_size + hidden_size

        # Eq(9): Le — shared, maps to N_QUBITS
        self.Le = nn.Linear(combined, N_QUBITS)

        # VQC trainable weights — one per gate
        vqc_shape = (N_LAYERS, N_QUBITS * 2)
        self.vqc_weights = nn.ParameterDict({
            'forget': nn.Parameter(torch.randn(vqc_shape) * 0.1),
            'input' : nn.Parameter(torch.randn(vqc_shape) * 0.1),
            'cell'  : nn.Parameter(torch.randn(vqc_shape) * 0.1),
            'output': nn.Parameter(torch.randn(vqc_shape) * 0.1),
        })

        # Eq(10-14): Li — separate per gate
        self.L1 = nn.Linear(N_QUBITS, hidden_size)  # forget
        self.L2 = nn.Linear(N_QUBITS, hidden_size)  # input
        self.L3 = nn.Linear(N_QUBITS, hidden_size)  # cell
        self.L4 = nn.Linear(N_QUBITS, hidden_size)  # output

    def _vqc(self, z, gate):
        w = self.vqc_weights[gate]
        # .float() casts float64 (qiskit.aer expectation values) → float32 (nn.Linear we
```

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

```
        return torch.stack([torch.stack(vqc_scz(z[i], w)) for i in range(z.shape[0])]).f

    def forward(self, x_t, h_prev, c_prev):
        v_t = torch.cat([h_prev, x_t], dim=-1)      # concatenate
        z_t = torch.tanh(self.Le(v_t))              # Eq(9): shared Le
        q_f = self._vqc(z_t, 'forget')
        q_i = self._vqc(z_t, 'input')
        q_g = self._vqc(z_t, 'cell')
        q_o = self._vqc(z_t, 'output')
        f_t = torch.sigmoid(self.L1((q_f+1)/2))     # Eq(10)
        i_t = torch.sigmoid(self.L2((q_i+1)/2))     # Eq(11)
        g_t = torch.tanh   (self.L3(q_g))           # Eq(12)
        o_t = torch.sigmoid(self.L4((q_o+1)/2))     # Eq(14)
        c_t = f_t * c_prev + i_t * g_t             # Eq(13)
        h_t = o_t * torch.tanh(c_t)                # Eq(15)
        return h_t, c_t

_c = LQLSTMCell(20, N_QUBITS)
_q = sum(p.numel() for n,p in _c.named_parameters() if 'vqc' in n)
_l = sum(p.numel() for n,p in _c.named_parameters() if 'vqc' not in n)
print(f"Cell — quantum: {_q}  classical: {_l}  total: {_q+_l}")
print(f"Paper Table 1: 205 total (hidden=4, qubits=4, input=9)")
```

```
Cell — quantum: 64  classical: 180  total: 244
Paper Table 1: 205 total (hidden=4, qubits=4, input=9)

Paper Table 1: 205 total (hidden=4, qubits=4, input=9)
```

Section 6 — Bidirectional L-QLSTM + 5-Machine Output Model

In [7]:

```
class BiLQLSTM(nn.Module):
    """Bidirectional L-QLSTM — extension of paper's unidirectional model."""
    def __init__(self, input_size, hidden_size):
        super().__init__()
        self.hs  = hidden_size
        self.fwd = LQLSTMCell(input_size, hidden_size)
        self.bwd = LQLSTMCell(input_size, hidden_size)

    def forward(self, x):
        B = x.size(0)
        hf=cf=torch.zeros(B,self.hs)
        hb=cb=torch.zeros(B,self.hs)
        for t in range(x.size(1)):
            hf,cf = self.fwd(x[:,t,:],hf,cf)
        for t in reversed(range(x.size(1))):
            hb,cb = self.bwd(x[:,t,:],hb,cb)
        return torch.cat([hf,hb],dim=-1)

class FleetLQLSTM(nn.Module):
    """Fleet model: BiLQLSTM + 5 Lo heads (Eq.16 per machine)."""
    def __init__(self, input_size, hidden_size, n_machines):
        super().__init__()
        self.encoder = BiLQLSTM(input_size, hidden_size)
        self.heads = nn.ModuleList([
            nn.Sequential(nn.Linear(2*hidden_size,8),nn.ReLU(),nn.Linear(8,1),nn.Sigmoid
            for _ in range(n_machines)
```

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

```
        ])
    def forward(self, x):
        s = self.encoder(x)
        return torch.cat([h(s) for h in self.heads], dim=-1)

INPUT_SIZE  = X_tr.shape[2]
HIDDEN_SIZE = N_QUBITS
N_MACHINES  = len(MACHINES)

model = FleetLQLSTM(INPUT_SIZE, HIDDEN_SIZE, N_MACHINES)
tot = sum(p.numel() for p in model.parameters() if p.requires_grad)
qp  = sum(p.numel() for n,p in model.named_parameters() if 'vqc' in n)
print(f"Total: {tot}  |  Quantum: {qp} ({100*qp//tot}%)  |  Classical: {tot-qp} ({100*(t
```

```
Total: 893  |  Quantum: 128 (14%)  |  Classical: 765 (85%)
```

Section 7 — Training (Paper Protocol: MSE, Adam lr=0.001, Early Stop)

In [8]:

```
N_EPOCHS_MAX=30; N_EPOCHS_MIN=5; BATCH_SIZE=4; LR=0.001; PATIENCE=5

tr_dl = DataLoader(TensorDataset(torch.tensor(X_tr),torch.tensor(y_tr)),batch_size=BATCH
va_dl = DataLoader(TensorDataset(torch.tensor(X_va),torch.tensor(y_va)),batch_size=BATCH
te_dl = DataLoader(TensorDataset(torch.tensor(X_te),torch.tensor(y_te)),batch_size=BATCH

criterion = nn.MSELoss()
optimizer = torch.optim.Adam(model.parameters(), lr=LR)

tr_losses,va_losses=[],[]
best_val,pat,best_state=float('inf'),0,None

for epoch in range(1, N_EPOCHS_MAX+1):
    model.train()
    ep=0.0
    for xb,yb in tr_dl:
        optimizer.zero_grad()
        loss=criterion(model(xb),yb)
        loss.backward(); optimizer.step(); ep+=loss.item()
    avg_tr=ep/len(tr_dl); tr_losses.append(avg_tr)

    model.eval()
    with torch.no_grad():
        avg_va=sum(criterion(model(xb),yb).item() for xb,yb in va_dl)/len(va_dl)
    va_losses.append(avg_va)

    mark=""
    if avg_va < best_val-1e-5:
        best_val=avg_va
        best_state={k:v.clone() for k,v in model.state_dict().items()}
        pat=0; mark=" * best"
    else:
        pat+=1
        if epoch>=N_EPOCHS_MIN and pat>=PATIENCE:
            print(f"Early stop ep{epoch}. Best val: {best_val:.5f}"); break

    print(f"Ep {epoch:3d}/{N_EPOCHS_MAX} | train={avg_tr:.5f} val={avg_va:.5f}{mark}")
```

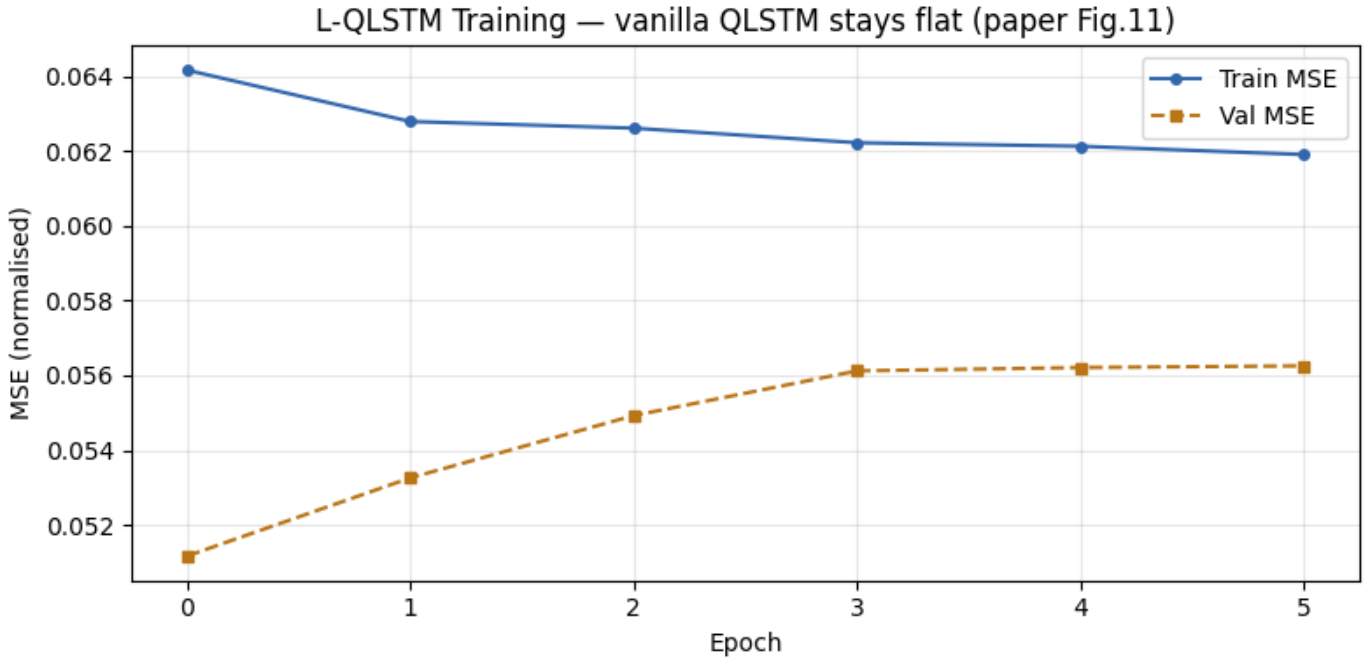
Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

```
if best_state: model.load_state_dict(best_state)
print("Training complete.")
```

```
Ep   1/30 | train=0.06416 val=0.05117 * best
Ep   2/30 | train=0.06278 val=0.05326
Ep   3/30 | train=0.06261 val=0.05492
Ep   4/30 | train=0.06222 val=0.05612
Ep   5/30 | train=0.06212 val=0.05621
Early stop ep6. Best val: 0.05117
Training complete.
```

In [9]:

```
fig,ax=plt.subplots(figsize=(8,4))
ax.plot(tr_losses,'o-',color='#3266ad',label='Train MSE',lw=1.5,ms=4)
ax.plot(va_losses,'s--',color='#BA7517',label='Val MSE',lw=1.5,ms=4)
ax.set_xlabel('Epoch'); ax.set_ylabel('MSE (normalised)')
ax.set_title('L-QLSTM Training — vanilla QLSTM stays flat (paper Fig.11)')
ax.legend(); ax.grid(True,alpha=0.3); plt.tight_layout(); plt.show()
```



Section 8 — Evaluation: RMSE and MAE (Paper Table 2 Metrics)

In [10]:

```
model.eval()
pn,tn=[],[]
with torch.no_grad():
    for xb,yb in te_dl: pn.append(model(xb).numpy()); tn.append(yb.numpy())
pn=np.concatenate(pn); tn=np.concatenate(tn)

rmse_n=np.sqrt(mean_squared_error(tn,pn))
mae_n=mean_absolute_error(tn,pn)
print("Test (normalised — paper scale):")
print(f"  RMSE: {rmse_n:.4f}  |  Paper L-QLSTM: 0.0436  |  Paper QLSTM: 0.3024")
print(f"  MAE : {mae_n:.4f}  |  Paper L-QLSTM: 0.0322  |  Paper QLSTM: 0.2723")

pr=scaler_y.inverse_transform(pn); tr=scaler_y.inverse_transform(tn)
print(f"\nTest (real CO2e kg/day):")
```

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

```
print(f"  RMSE: {np.sqrt(mean_squared_error(tr,pr)):.2f} kg/day")
print(f"  MAE : {mean_absolute_error(tr,pr):.2f} kg/day")
print("\nPer-machine MAE:")
for i,m in enumerate(MACHINE_LIST):
    print(f"  {m:<18}: {mean_absolute_error(tr[:,i],pr[:,i]):.2f} kg/day")
```

```
Test (normalised — paper scale):
  RMSE: 0.2313  |  Paper L-QLSTM: 0.0436  |  Paper QLSTM: 0.3024
  MAE : 0.1889  |  Paper L-QLSTM: 0.0322  |  Paper QLSTM: 0.2723

Test (real CO2e kg/day):
  RMSE: 222.89 kg/day
  MAE : 177.34 kg/day

Per-machine MAE:
  Excavator_A       : 131.47 kg/day
  Generator_B       : 176.96 kg/day
  Crane_C           : 208.81 kg/day
  Compressor_D      : 127.29 kg/day
  Truck_Fleet       : 242.15 kg/day
```

Section 9 — 30 / 60 / 90 Day Forecast and Emissions Report

In [11]:

```
def autoregressive_forecast(model, X_sc, scaler_y, lb, n):
    model.eval(); w=X_sc[-lb:].copy(); out=[]
    with torch.no_grad():
        for _ in range(n):
            p=model(torch.tensor(w[np.newaxis],dtype=torch.float32)).squeeze(0).numpy()
            out.append(p); w=np.vstack([w[1:],w[-1]])
    return scaler_y.inverse_transform(np.array(out))

forecasts={h:autoregressive_forecast(model,X_sc,scaler_y,LOOKBACK,h) for h in [30,60,90]

for h in [30,60,90]:
    pred=forecasts[h]; totals=pred.sum(axis=0); fleet=totals.sum()
    rows=[]
    for i,m in enumerate(MACHINE_LIST):
        rows.append({'Machine':m,'CO2e (kg)':round(totals[i],1),'Tonnes':round(totals[i]
                     'Fleet %':round(100*totals[i]/fleet,1),'Daily avg':round(totals[i]/
    rows.append({'Machine':'FLEET TOTAL','CO2e (kg)':round(fleet,1),'Tonnes':round(fleet
                 'Fleet %':100.0,'Daily avg':round(fleet/h,1)})
    print(f"\n{'='*55}  {h}-DAY FORECAST")
    print(pd.DataFrame(rows).to_string(index=False))
```

```
=======================================================  30-DAY FORECAST
     Machine    CO2e (kg)  Tonnes  Fleet %  Daily avg
 Excavator_A 12029.500000  12.029     15.3      401.0
 Generator_B 14575.700195  14.576     18.6      485.9
     Crane_C 19820.000000  19.820     25.3      660.7
Compressor_D 12097.700195  12.098     15.4      403.3
 Truck_Fleet 19929.800781  19.930     25.4      664.3
 FLEET TOTAL 78452.601562  78.453    100.0     2615.1

=======================================================  60-DAY FORECAST
     Machine     CO2e (kg)  Tonnes  Fleet %  Daily avg
 Excavator_A  24058.400391  24.058     15.3      401.0
 Generator_B  29150.400391  29.150     18.6      485.8
```

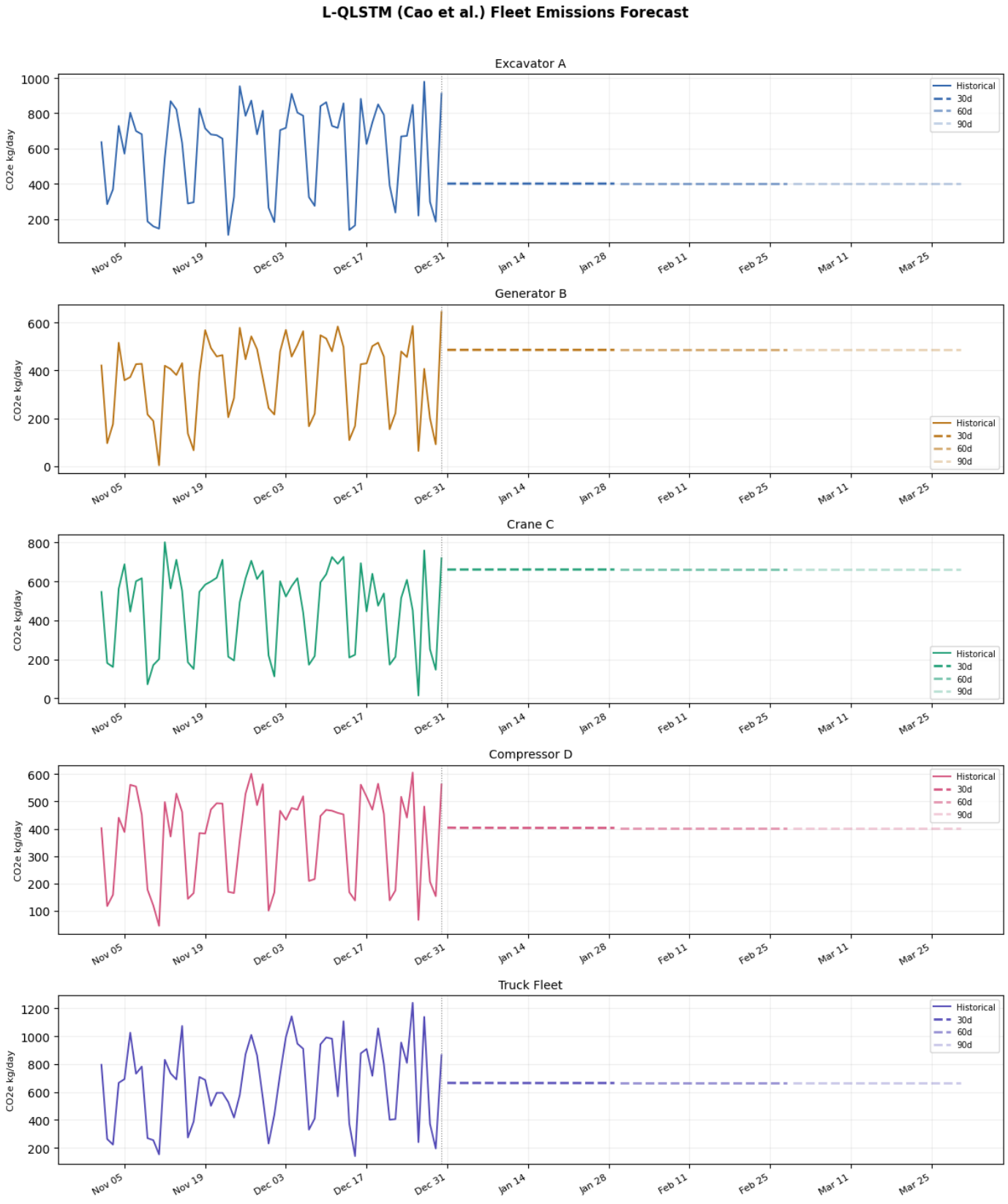
Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

```
     Crane_C  39641.101562  39.641     25.3      660.7
Compressor_D  24191.300781  24.191     15.4      403.2
 Truck_Fleet  39856.000000  39.856     25.4      664.3
 FLEET TOTAL 156897.203125 156.897    100.0     2615.0

=========================================================  90-DAY FORECAST
     Machine     CO2e (kg)  Tonnes  Fleet %  Daily avg
 Excavator_A  36087.300781  36.087     15.3      401.0
 Generator_B  43725.101562  43.725     18.6      485.8
     Crane_C  59462.199219  59.462     25.3      660.7
Compressor_D  36285.000000  36.285     15.4      403.2
 Truck_Fleet  59782.199219  59.782     25.4      664.2
 FLEET TOTAL 235341.703125 235.342    100.0     2614.9
```

In [12]:

```
hist=df.pivot(index='date',columns='machine',values='co2e_kg')
hist.index=pd.to_datetime(hist.index)
last60=hist.iloc[-60:]; last_d=hist.index[-1]
fc_d=[last_d+timedelta(days=i+1) for i in range(90)]
colors=['#3266ad','#BA7517','#1D9E75','#D4537E','#534AB7']

fig,axes=plt.subplots(len(MACHINES),1,figsize=(12,14))
fig.suptitle('L-QLSTM (Cao et al.) Fleet Emissions Forecast',fontsize=12,fontweight='bol
for i,(ax,m) in enumerate(zip(axes,MACHINE_LIST)):
    c=colors[i]; f=forecasts[90][:,i]
    ax.plot(last60.index,last60[m].values,color=c,lw=1.4,label='Historical')
    ax.plot(fc_d[:30],f[:30],color=c,lw=2,ls='--',alpha=0.95,label='30d')
    ax.plot(fc_d[30:60],f[30:60],color=c,lw=2,ls='--',alpha=0.60,label='60d')
    ax.plot(fc_d[60:],f[60:],color=c,lw=2,ls='--',alpha=0.30,label='90d')
    ax.axvline(last_d,color='gray',lw=0.8,ls=':')
    ax.set_title(m.replace('_',' '),fontsize=10); ax.set_ylabel('CO2e kg/day',fontsize=8
    ax.legend(fontsize=7); ax.grid(True,alpha=0.2)
    ax.xaxis.set_major_formatter(mdates.DateFormatter('%b %d'))
    ax.xaxis.set_major_locator(mdates.WeekdayLocator(interval=2))
    plt.setp(ax.get_xticklabels(),rotation=30,ha='right',fontsize=8)
plt.tight_layout(); plt.show()
```

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF



Summary

Component	Paper	Code
Shared Le before VQCs	Eq(9), Sec 3.2	`nn.Linear(combined, N_QUBITS)`
Separate Li after VQCs	Eq(10-14), Sec 3.2	`4 x nn.Linear(N_QUBITS, hidden)`

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

Component	Paper	Code
SCZ variational layer	Sec 3.3, Fig 8	`CRZ` all-to-all + `RZ`
MSE + Adam lr=0.001	Sec 4.1	Exact match
64/16/20 split + early stop	Sec 4.1	Exact match
RMSE + MAE evaluation	Sec 4.1.2	sklearn
Bidirectional *(our ext.)*	—	Fwd + bwd LQLSTMCell
5-machine output *(our ext.)*	—	5 x Lo head (Eq.16)

Run on real IBM hardware

```
from qiskit_ibm_runtime import QiskitRuntimeService
service = QiskitRuntimeService(channel='ibm_quantum', token='YOUR_TOKEN')
backend = service.least_busy(operational=True, simulator=False)
dev = qml.device('qiskit.remote', wires=N_QUBITS, backend=backend)

# With noise model:
from qiskit_aer.noise import NoiseModel
noise = NoiseModel.from_backend(backend)
dev = qml.device('qiskit.aer', wires=N_QUBITS, noise_model=noise)
```

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF